DECLARATION OF AUTHENTICITY

I, Tiaan Gerber ,hereby declare that the contents of this assessment is entirely my own work, completed by me without any paraphrasing or copying, or presented as my own work accessed from any AI Apps, for example ChatGPT, Co-pilot, Perplexity, or any other App. I have read and understood the Examination Rules and Regulations.

Signature: Tiaan Gerber

Date: 2026/04/23

PROJECT OVERVIEW

Project Title: Smart Student Profile Processor

Module: Software Design and Testing C# (SDT621)

NQF Level: 6

Developer: [Your Full Name] Tiaan Geber

Purpose of the Project

This project was developed as a real-world workplace simulation where a Junior Software Developer is required to build a student information processing system. The system allows training providers to capture student information, validate all data entries, calculate student readiness indicators, display structured outputs, and prepare student data for future database storage.

What the System Does

The Smart Student Profile Processor captures the following student information:

- Full name
- Age
- Course selection (Programming, Data Science, or Cyber Security)
- Three test scores
- Attendance percentage
- Technical self-assessment score

The system then calculates a Readiness Score for each student using the following formula:

- Test average contributes 50 percent
- Attendance contributes 30 percent
- Technical assessment contributes 20 percent

Based on the final score, the student is classified as:

- Ready for scores between 80 and 100 percent
- Partially ready for scores between 60 and 79 percent
- Not ready for scores below 60 percent

The system also stores student data permanently using a CSV file, so information is not lost when the application is closed.

CONSOLE APPLICATION

Overview

The Console Application is a text-based version of the Smart Student Profile Processor. It runs in a terminal window and provides a menu-driven interface for users who prefer keyboard navigation.

Key Features

Main Menu Options

The application presents a menu with six options: Add New Student, View Current Student Profile, Calculate Readiness Score, Generate Database Report, Clear All Data, and Exit.

Add New Student Feature

When the user selects Add New Student, the system automatically generates a unique student ID. The user is then prompted to enter the student's full name, age, course choice, three test scores, attendance percentage, and technical self-assessment level. Each input is validated immediately. Names must be at least three characters long. Age must be between 16 and 100. Test scores and attendance must be between 0 and 100. The technical assessment must be between 1 and 5.

Student Profile Display

The View Current Student Profile option displays all stored information for the most recently added student. The output is formatted as a clean table showing student ID, name, age, course, individual test scores, test average, attendance, technical level, and readiness score.

Readiness Score Calculation

The Calculate Readiness Score option shows a detailed breakdown of how the readiness score was calculated. It displays the test average with its 50 percent weight, the attendance score with its 30 percent weight, and the technical score with its 20 percent weight. The final score is then shown along with the classification level.

Database Report Generation

The Generate Database Report option prepares student data in a JSON-like format that is ready to be inserted into a SQL or NoSQL database. This demonstrates understanding of how applications interface with database systems.

Data Persistence

The application automatically saves student data to a CSV file whenever a new student is added. When the application starts again, it loads any existing data from this file.

Learning Outcomes Demonstrated in Console Application

The Console Application demonstrates the structure of a C# program with proper namespace and class organization. It uses variables and data types including string, int, double, and DateTime. Identifiers follow PascalCase for methods and camelCase for local variables. Arithmetic operators are used in the readiness score calculation. Expressions include the calculated test average property. Methods are created for adding students, validating input, calculating readiness, and displaying reports. Method parameters are used to pass data between methods. Method overloading is demonstrated with three versions of the CalculateReadiness method. Scope is shown through class-level variables and method-level local variables. Type casting is used when converting the technical assessment from a 1-to-5 scale to a 0-to-100 scale. Console input and output are used throughout for user interaction.

```
Smart Student Profile Processor v2.0
Kitumaini Digital Solutions | Professional Edition
============================================================
Students in system: 0
============================================================

[ MAIN MENU ]
1. Add New Student
2. View All Students (Summary)
3. View Student Details
4. Readiness Report & Analytics
5. Export to Database Format
6. Save & Exit

Your choice:
```

```
[ ADD NEW STUDENT ]
----------------------------------------
Full Name (min 3 chars): Tiaan Gerber
Age (16-100): 18

Courses: 1=Programming | 2=Data Science | 3=Cyber Security
Select course (1-3): 2
Test Score 1 (0-100): 90
Test Score 2 (0-100): 70
Test Score 3 (0-100): 88
Attendance % (0-100): 94
Tech Self-Assessment (1-5): 4

[SUCCESS] Student added with ID: 8451
Standard Readiness: 85,5%
Weight-Adjusted (60/20/20): 84,4%

Press any key to continue...
```

```
Smart Student Profile Process

[ ADD NEW STUDENT ]
----------------------------------------
Full Name (min 3 chars): Jaco Gerber
Age (16-100): 22

Courses: 1=Programming | 2=Data Science | 3=Cyber Security
Select course (1-3): 3
Test Score 1 (0-100): 80
Test Score 2 (0-100): 90
Test Score 3 (0-100): 96
Attendance % (0-100): 98
Tech Self-Assessment (1-5): 5

[SUCCESS] Student added with ID: 2939
Standard Readiness: 93,7%
Weight-Adjusted (60/20/20): 92,8%

Press any key to continue...

[ MAIN MENU ]
1. Add New Student
2. View All Students (Summary)
3. View Student Details
4. Readiness Report & Analytics
5. Export to Database Format
6. Save & Exit

Your choice: |
```

```
Smart Student Profile Process

[ ALL STUDENTS - SUMMARY VIEW ]
=====================================================================
| ID     | Name                 | Age | Course          | Avg     | Ready  |
---------------------------------------------------------------------
| 8451   | Tiaan Gerber         | 18  | Data Science    | 82,7    | 85,5   |
| 2939   | Jaco Gerber          | 22  | Cyber Security  | 88,7    | 93,7   |
---------------------------------------------------------------------
Total Students: 2

Press any key to continue...
```

WINDOWS FORMS APPLICATION

Overview

The Windows Forms Application is a graphical user interface version of the Smart Student Profile Processor. It provides a modern, dark-themed interface with tabs, buttons, text boxes, and data grids for a more intuitive user experience.

Key Features

Three-Tab Interface

The application is organized into three tabs: Add Student, Student List, and Analytics.

Add Student Tab

This tab contains three grouped sections for student information, test scores, and assessment. The student information section collects the full name, age, and course selection. The test scores section collects three test scores and attendance percentage. The assessment section collects the technical self-assessment level and contains the Add Student and Clear Form buttons.

Real-time validation is performed when the user clicks Add Student. Error messages appear in red next to the relevant fields. If validation passes, the student is added to the system, saved to the CSV file, and a success message is shown.

Student List Tab

This tab displays all students in a DataGridView with columns for ID, name, age, course, test average, readiness score, and level. A search panel at the top allows users to search by student name or ID. The search updates in real time as the user types. Buttons are provided to refresh the list, delete the selected student, and export data to the clipboard.

A status bar at the bottom of the window shows the current operation status and student count.

Analytics Tab

This tab provides statistical insights about all students in the system. It displays the total number of students, the average readiness score across all students, and the count of students in each readiness category. A progress bar visually represents the average readiness score. The top performer is highlighted with the highest readiness score.

Database Export Feature

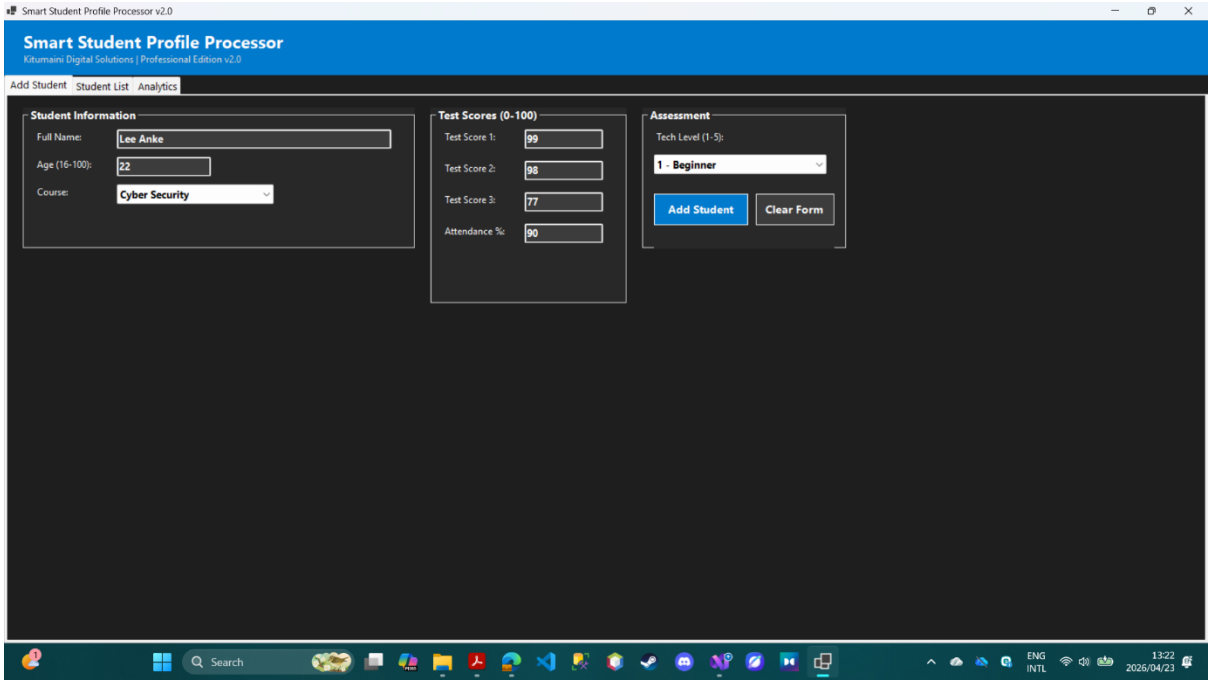
The Export to DB button copies student data to the clipboard in JSON format, making it ready for insertion into a database or API. The export includes student ID, name, age, course, test average, attendance, readiness score, and readiness level.

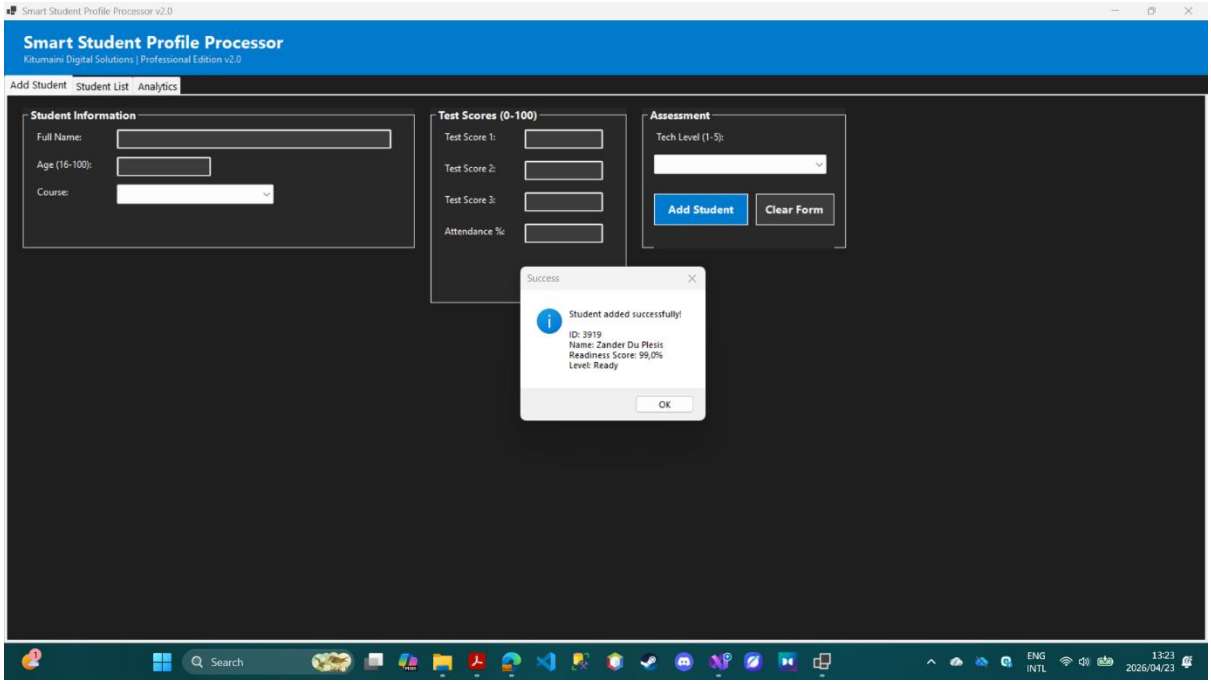
Data Persistence

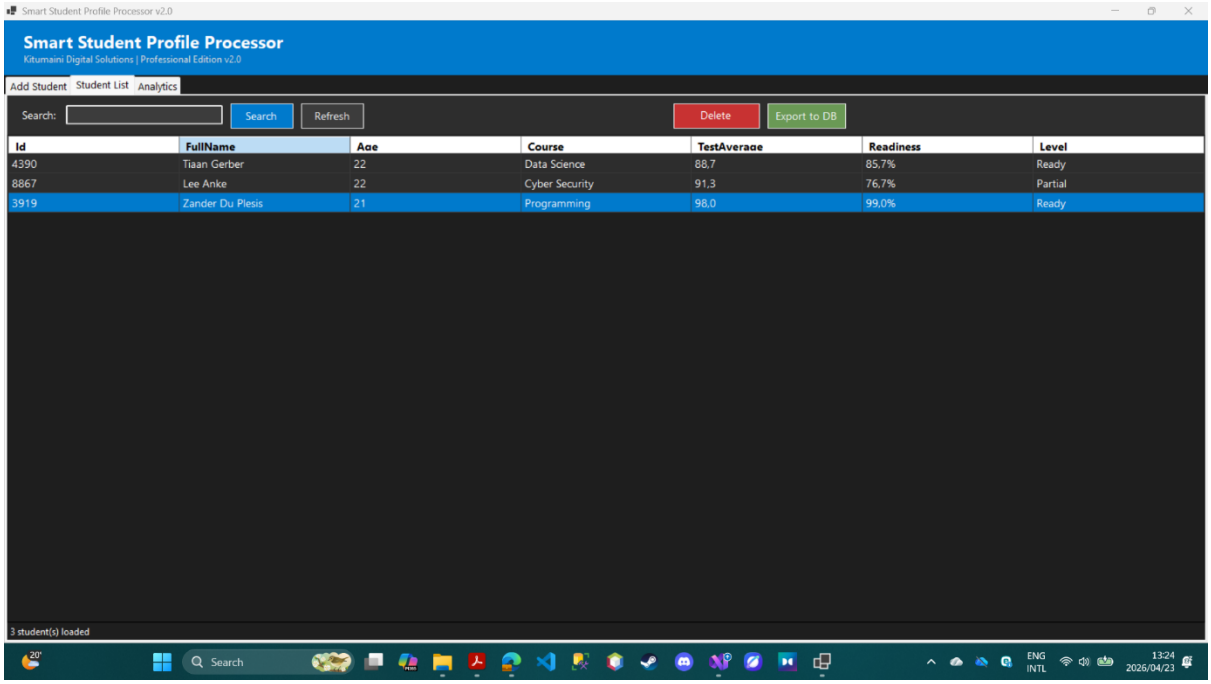
Like the Console Application, the Windows Forms application automatically saves data to a CSV file and loads existing data on startup.

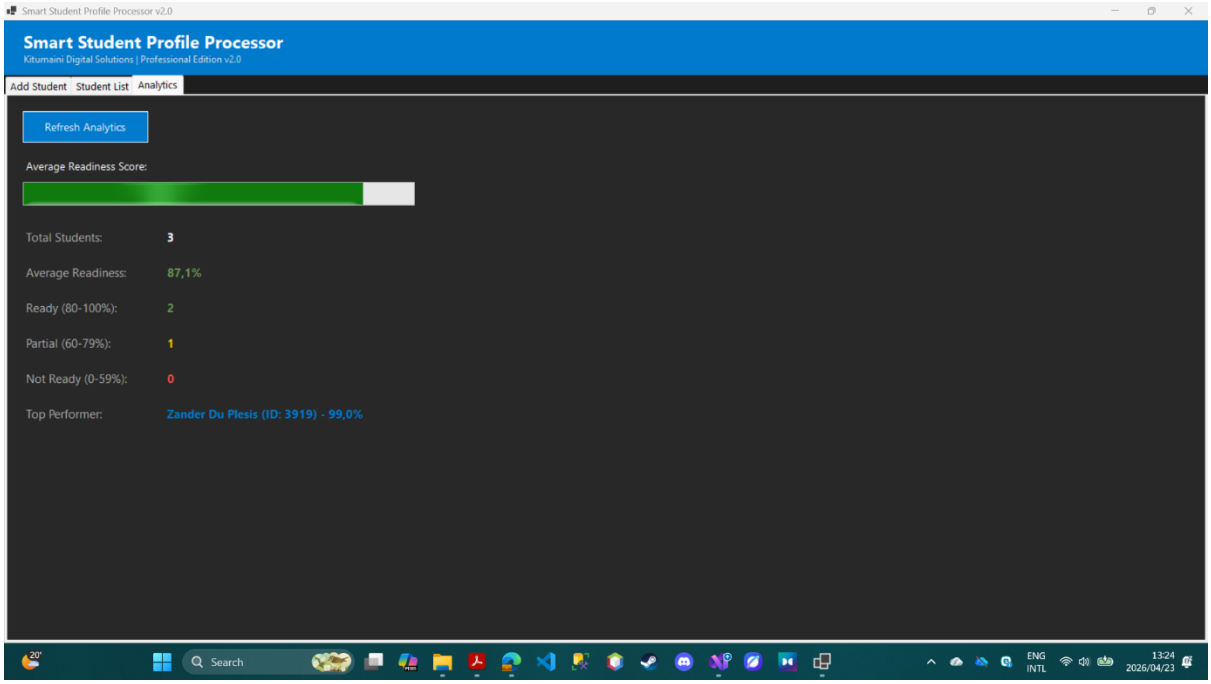
Learning Outcomes Demonstrated in Windows Forms Application

The Windows Forms Application demonstrates the same core C# concepts as the console version but adds Windows Forms interface basics. Controls such as TextBox, ComboBox, Button, DataGridView, ProgressBar, TabControl, GroupBox, Label, and StatusStrip are all used. Event handling is demonstrated through Click events for buttons and TextChanged events for the search box. Form layout is managed through the Designer file with explicit positioning. The same Student class is reused, demonstrating code reusability.


Smart Student Profile Processor v2.0
Smart Student Profile Processor
Kitumaini Digital Solutions | Professional Edition v2.0
Add Student
Student List
Analytics
Student Information
Full Name:
Lee Anke
Age (16-100):
22
Course:
Cyber Security
Test Scores (0-100)
Test Score 1:
99
Test Score 2:
98
Test Score 3:
77
Attendance %:
90
Assessment
Tech Level (1-5):
1 - Beginner
Add Student
Clear Form
Search
ENG
INTL
13:22
2026/04/23


Smart Student Profile Processor v2.0
Smart Student Profile Processor
Kitumaini Digital Solutions | Professional Edition v2.0
Add Student
Student List
Analytics
Student Information
Full Name:
Age (16-100):
Course:
Test Scores (0-100)
Test Score 1:
Test Score 2:
Test Score 3:
Attendance %:
Assessment
Tech Level (1-5):
Add Student
Clear Form
Success
Student added successfully!
ID: 3919
Name: Zander Du Plesis
Readiness Score: 99,0%
Level: Ready
OK
Search
ENG
INTL
13:23
2026/04/23


Smart Student Profile Processor v2.0
Smart Student Profile Processor
Kitumaini Digital Solutions | Professional Edition v2.0
Add Student
Student List
Analytics
Search:
Search
Refresh
Delete
Export to DB
Id | FullName | Age | Course | TestAverage | Readiness | Level
4390 | Tiaan Gerber | 22 | Data Science | 88,7 | 85,7% | Ready
8867 | Lee Anke | 22 | Cyber Security | 91,3 | 76,7% | Partial
3919 | Zander Du Plesis | 21 | Programming | 98,0 | 99,0% | Ready
3 student(s) loaded


Smart Student Profile Processor v2.0
Smart Student Profile Processor
Kitumaini Digital Solutions | Professional Edition v2.0
Add Student
Student List
Analytics
Refresh Analytics
Average Readiness Score:
Total Students: 3
Average Readiness: 87,1%
Ready (80-100%): 2
Partial (60-79%): 1
Not Ready (0-59%): 0
Top Performer: Zander Du Plesis (ID: 3919) - 99,0%

GITHUB SUBMISSION

Repository Information

Repository Link:

Example Format:

Repository Contents

The GitHub repository contains the complete solution with all project files. The solution includes both the Console Application project and the Windows Forms Application project. The Student class file is shared between both projects. The repository builds successfully with no errors. All code files are included and match the work submitted in the ZIP file.

GitHub Evidence

CHALLENGES AND SOLUTIONS

Challenge 1: Data Persistence

Problem: Student data was lost every time the application closed.

Solution: Implemented CSV file save and load functionality using StreamReader and StreamWriter. Data is saved after every add or delete operation and loaded when the application starts.

Challenge 2: Windows Forms Layout Overlap

Problem: Controls were overlapping and not aligning properly on the form.

Solution: Used GroupBox containers to organize related controls and applied explicit X and Y coordinates instead of relying on DockStyle. This gave precise control over the layout.

Challenge 3: Search Functionality

Problem: Needed to filter the DataGridView based on user input.

Solution: Used LINQ Where clause to filter the student list by name or ID, then created a new anonymous object list to refresh the DataGridView.

Challenge 4: Method Overloading Demonstration

Problem: Needed to clearly show method overloading with meaningful differences.

Solution: Created three versions of CalculateReadiness. The first uses default weights. The second accepts custom weight parameters. The third is a static helper method for quick calculations.

Challenge 5: Input Validation

Problem: Users could enter invalid data such as negative ages or test scores above 100.

Solution: Implemented validation loops in the Console application that repeat until valid data is entered. In the Windows Forms application, validation runs on button click with error labels shown next to invalid fields.